

Project: Healthcare - Persistency of a Drug (Data Science Specialization)

Deliverables - Week 13 (Final Project)

Group Name: DataVision

Name: Magdalena Ivanova

University: IMC Krems

Email: magdalena.ivanova.ds@gmail.com

Country: Austria

Specialization: Data Science

Batch Code: LISUM49

Date: 24 November 2025

Submitted to: Data Glacier

Table of Contents

1. Problem Description
 2. Data Collection
 3. Data Preparation
 4. Analysis
 - 4.1 Numerical Analysis
 - 4.2 Patient Demographics
 - 4.3 Physician and Provider Analysis
 - 4.4 Risk and Adherence Factors
 5. Model Building and Evaluation
 6. Model Selection and Comparison
-

1. Problem Description

The pharmaceutical sector continues to struggle with understanding why some patients remain consistent with prescribed medication plans while others drop off. Patient non-persistency can lead to poor health results, higher costs, and added treatment complexity.

To address this issue, ABC Pharma Company initiated a project to uncover the underlying factors driving drug persistency using data-driven methods.

The dataset contains multiple attributes, including patient demographics, physician details, treatment and disease information, and adherence data.

The main objective is to analyze these factors and construct a model capable of predicting whether a patient will stay persistent (1) or discontinue the treatment (0).

2. Data Collection

The dataset, titled Healthcare_dataset.xlsx, includes 3,424 records and 27 variables describing patient profiles and treatment data.

No missing entries were identified, which simplified preprocessing.

```
# Check if there are any duplicate values:

duplicates = data.duplicated()

if duplicates.any():
    print("Duplicates exist in the DataFrame.")
else:
    print("No duplicates found in the DataFrame.")

143]

.. No duplicates found in the DataFrame.

# Double check if there are any null (NA) values:

Null_Values = data.isnull().sum()

if Null_Values.any():
    print("Null values exist in the DataFrame.")
else:
    print("No null values exist in the DataFrame.")

144]

.. No null values exist in the DataFrame.
```

3. Data Preparation

3.1 Data Transformation

Several categorical variables were labeled with 'Y' and 'N'.

These were converted to **1s and 0s** to standardize input and make them compatible for modeling and visualization.

Data Preparation

```
data = pd.read_csv('Healthcare.csv')
data.head(5)
```

t	...	Risk_Family_History_Of_Osteoporosis	Risk_Low_Calcium_Intake	Risk_Vitamin_D_Insufficiency	Risk_Poor_Health_Frailty	Risk_Excessive_Thinness
1	...	N	N	N	N	N
1	...	N	N	N	N	N
1	...	N	Y	N	N	N
1	...	N	N	N	N	N
1	...	N	N	N	N	N

```
data = data.replace('N', 0)
data = data.replace('Y', 1)
data.head()
```

...	Risk_Family_History_Of_Osteoporosis	Risk_Low_Calcium_Intake	Risk_Vitamin_D_Insufficiency	Risk_Poor_Health_Frailty	Risk_Excessive_Thinness
...	0	0	0	0	0
...	0	0	0	0	0
...	0	1	0	0	0
...	0	0	0	0	0
...	0	0	0	0	0

3.2 Data Verification

After converting categorical fields, data types were validated and duplicates were checked. The final dataset passed all quality checks and was ready for further analysis.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3424 entries, 0 to 3423
Data columns (total 69 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Ptid                                       3424 non-null   object
1   Persistence_Flag                         3424 non-null   object
2   Gender                                   3424 non-null   object
3   Race                                     3424 non-null   object
4   Ethnicity                               3424 non-null   object
5   Region                                   3424 non-null   object
6   Age_Bucket                              3424 non-null   object
7   Ntm_Speciality                           3424 non-null   object
8   Ntm_Specialist_Flag                     3424 non-null   object
9   Ntm_Speciality_Bucket                   3424 non-null   object
10  Gluco_Record_Prior_Ntm                  3424 non-null   int64
11  Gluco_Record_During_Rx                  3424 non-null   int64
12  Dexa_Freq_During_Rx                     3424 non-null   int64
13  Dexa_During_Rx                          3424 non-null   int64
14  Frag_Frac_Prior_Ntm                     3424 non-null   int64
15  Frag_Frac_During_Rx                     3424 non-null   int64
16  Risk_Segment_Prior_Ntm                   3424 non-null   object
17  Tscore_Bucket_Prior_Ntm                  3424 non-null   object
18  Risk_Segment_During_Rx                  3424 non-null   object
19  Tscore_Bucket_During_Rx                  3424 non-null   object
...
67  Risk_Recurring_Falls                    3424 non-null   int64
68  Count_Of_Risks                          3424 non-null   int64
dtypes: int64(52), object(17)
```

```
for column in data.columns:
    unique_values = data[column].unique()
    print("The unique values in column",column,"are:",unique_values)

[142]

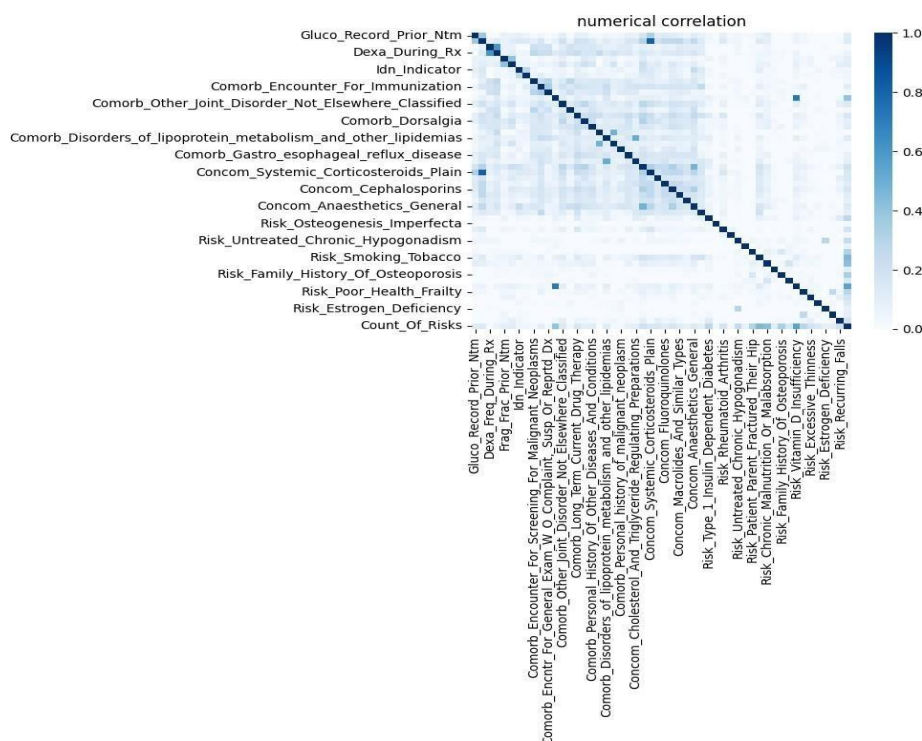
... The unique values in column Ptid are: ['P1' 'P2' 'P3' ... 'P3422' 'P3423' 'P3424']
The unique values in column Persistency.Flag are: ['Persistent' 'Non-Persistent']
The unique values in column Gender are: ['Male' 'Female']
The unique values in column Race are: ['Caucasian' 'Asian' 'Other/Unknown' 'African American']
The unique values in column Ethnicity are: ['Not Hispanic' 'Hispanic' 'Unknown']
The unique values in column Region are: ['West' 'Midwest' 'South' 'Other/Unknown' 'Northeast']
The unique values in column Age.Bucket are: ['>75' '55-65' '65-75' '<55']
The unique values in column Ntm.Speciality are: ['GENERAL PRACTITIONER' 'Unknown' 'ENDOCRINOLOGY' 'RHEUMATOLOGY'
'ONCOLOGY' 'PATHOLOGY' 'OBSTETRICS AND GYNECOLOGY'
'PSYCHIATRY AND NEUROLOGY' 'ORTHOPEDIC SURGERY'
'PHYSICAL MEDICINE AND REHABILITATION' 'SURGERY AND SURGICAL SPECIALTIES'
'PEDIATRICS' 'PULMONARY MEDICINE' 'HEMATOLOGY & ONCOLOGY' 'UROLOGY'
'PAIN MEDICINE' 'NEUROLOGY' 'RADIOLOGY' 'GASTROENTEROLOGY'
'EMERGENCY MEDICINE' 'PODIATRY' 'OPHTHALMOLOGY' 'OCCUPATIONAL MEDICINE'
'TRANSPLANT SURGERY' 'PLASTIC SURGERY' 'CLINICAL NURSE SPECIALIST'
'OTOLARYNGOLOGY' 'HOSPITAL MEDICINE' 'ORTHOPEDICS' 'NEPHROLOGY'
'GERIATRIC MEDICINE' 'HOSPICE AND PALLIATIVE MEDICINE'
'OBSTETRICS & OBSTETRICS & GYNECOLOGY & OBSTETRICS & GYNECOLOGY'
'VASCULAR SURGERY' 'CARDIOLOGY' 'NUCLEAR MEDICINE']
The unique values in column Ntm.Specialist.Flag are: ['Others' 'Specialist']
The unique values in column Ntm.Speciality.Bucket are: ['OB/GYN/Others/PCP/Unknown' 'Endo/Onc/Uro' 'Rheum']
The unique values in column Gluco.Record.Prior.Ntm are: [0 1]
The unique values in column Gluco.Record.During.Rx are: [0 1]
The unique values in column Dexa.Freq.During.Rx are: [ 0 2 7 3 5 20 13 1 6 12 4 10 25 11 18 21 15 28
22 37 14 8 9 17 81 42 16 30 19 45 27 24 58 26 23 33
...
The unique values in column Risk.Estrogen.Deficiency are: [0 1]
The unique values in column Risk.Immobilization are: [0 1]
The unique values in column Risk.Recurring.Falls are: [0 1]
The unique values in column Count.Of.Risks are: [0 2 1 3 4 5 6 7]
```

4. Analysis

A thorough data exploration was conducted to gain insight into the relationships between patient attributes and persistency outcomes. The analysis was split into four key sections.

4.1 Numerical Analysis

Quantitative variables were explored her using correlation.

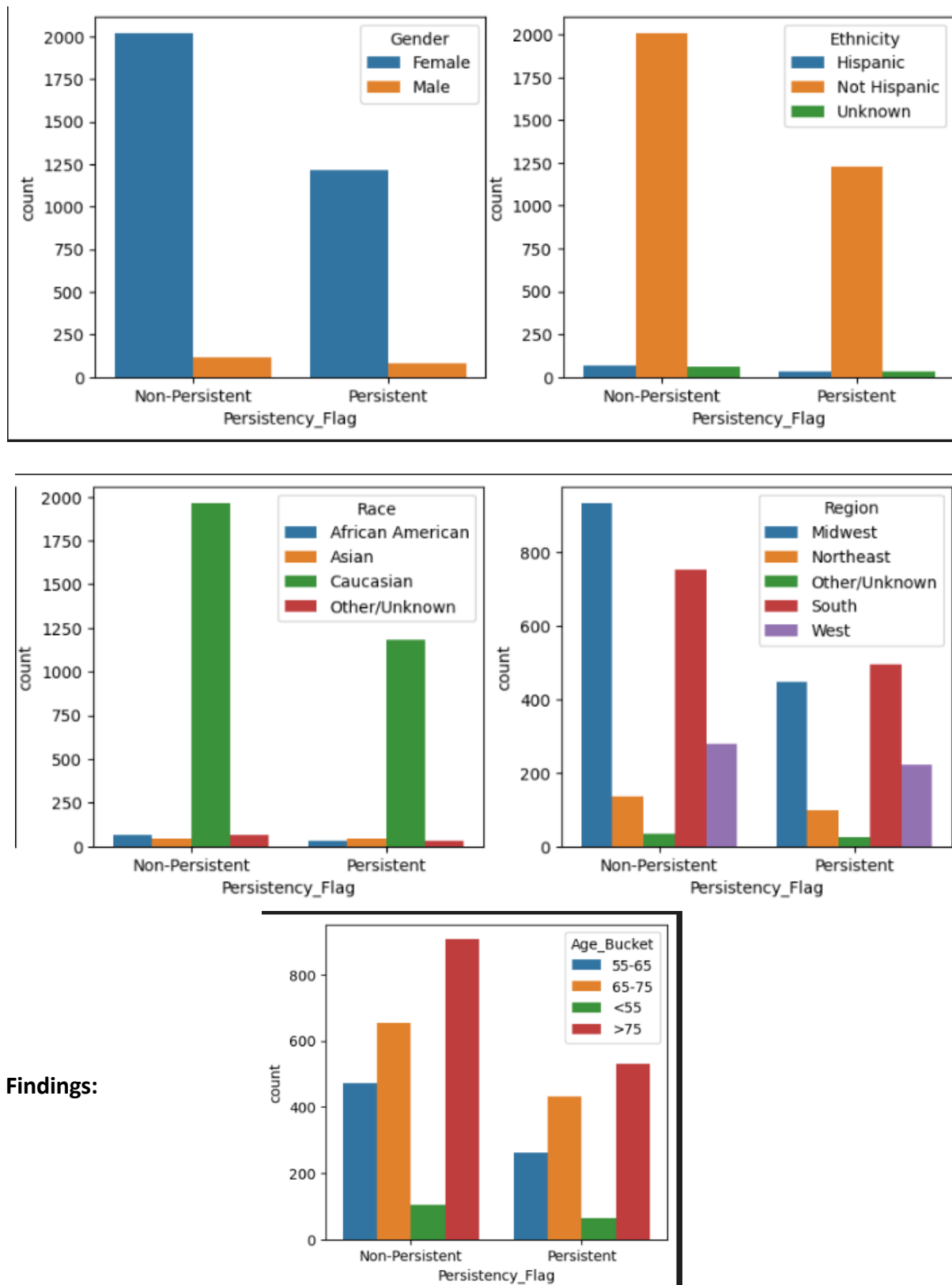


Findings:

- Correlation among numeric variables was rather weak, highlighting that categorical variables likely have greater influence.

4.2 Patient Demographics

Patients were categorized by Gender, Ethnicity, Race, Region, and Age Group to observe differences in persistency.



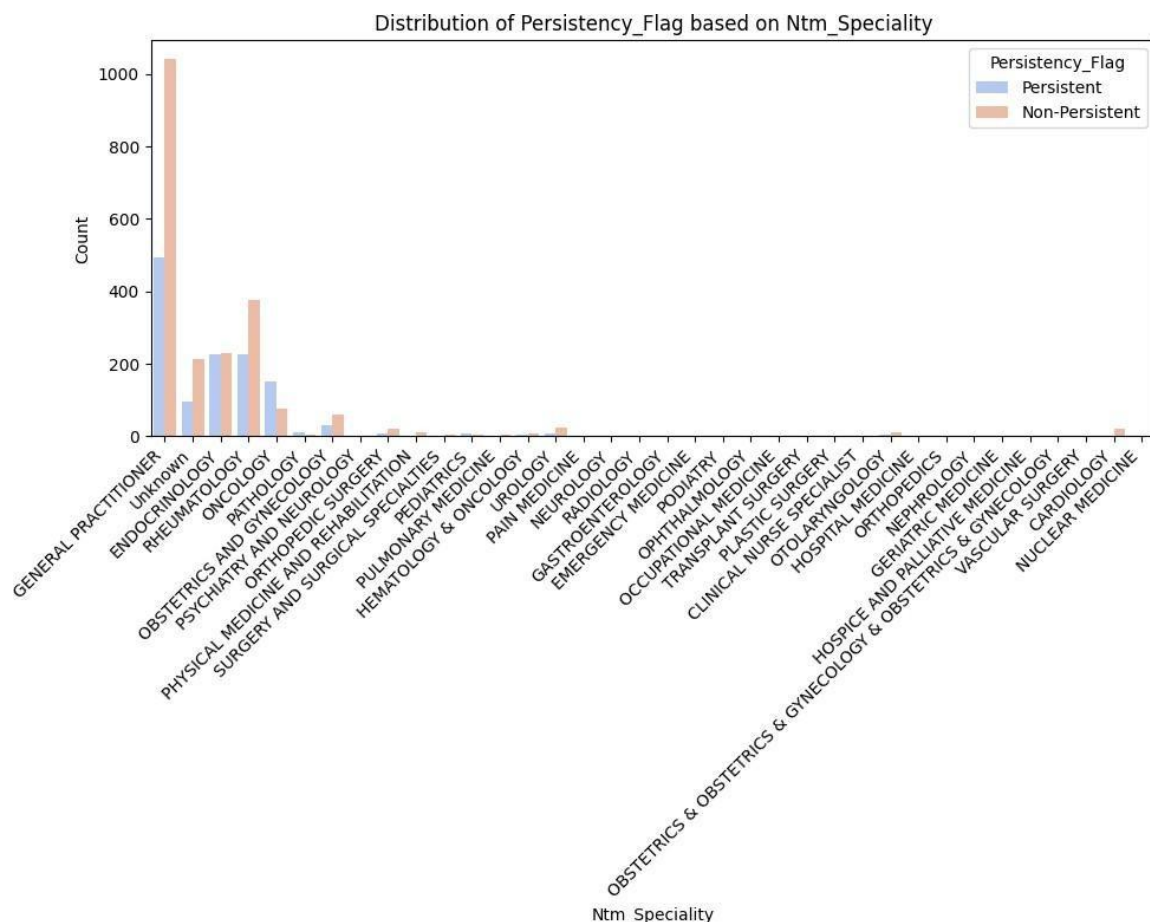
Findings:

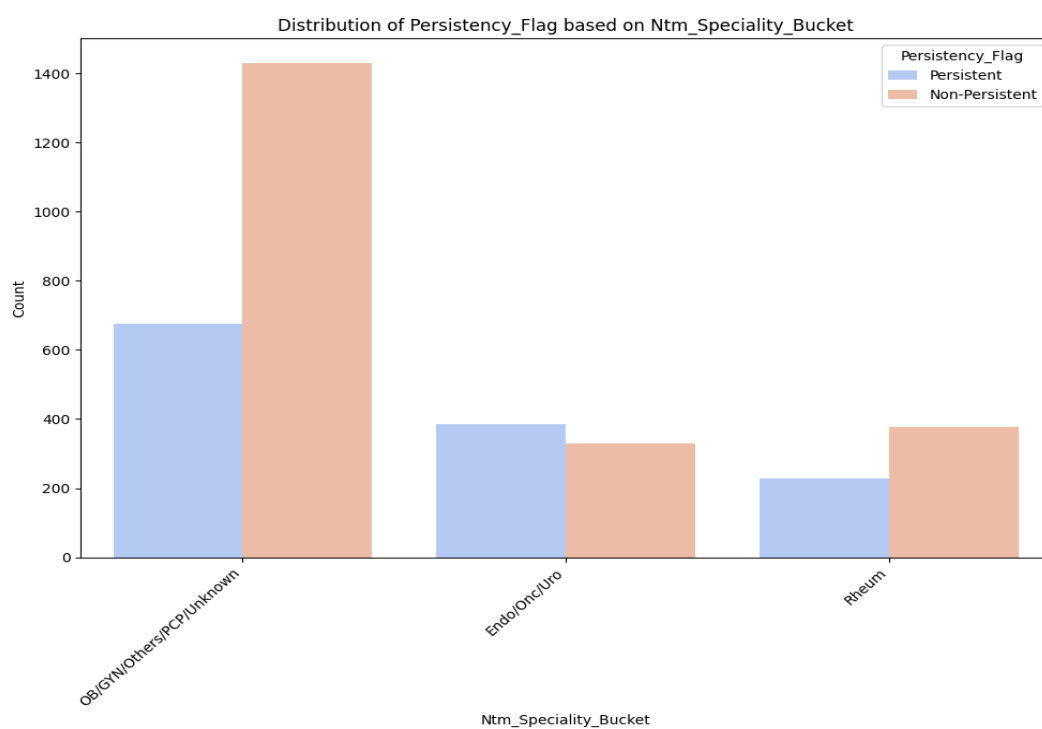
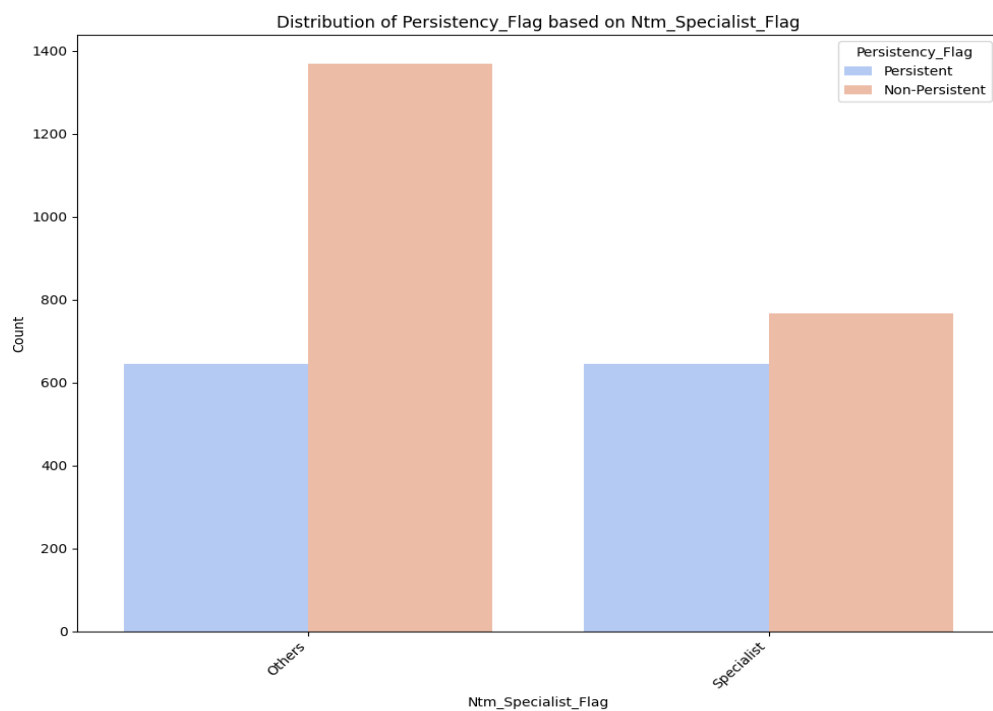
1. Female patients showed higher counts for both persistent and non-persistent cases, though non-persistence was slightly more frequent.
2. Caucasian and non-Hispanic groups represented the largest segments, with non-persistence again leading.
3. The South had the highest persistency rates, followed by Midwest and West.
4. Patients above 65 years old showed greater overall persistence.

4.3 Physician and Provider Analysis

This part focuses on the influence of physician attributes on persistency outcomes. The following features were analyzed:

- **Ntm_Specialty:** Physician's area of practice
- **Ntm_Specialist_Flag:** Indicates if the physician is a specialist
- **Ntm_Specialty_Bucket:** Categorized into OB/GYN/Others/PCP/Unknown, Endo/Onc/Uro, and Rheu





Findings:

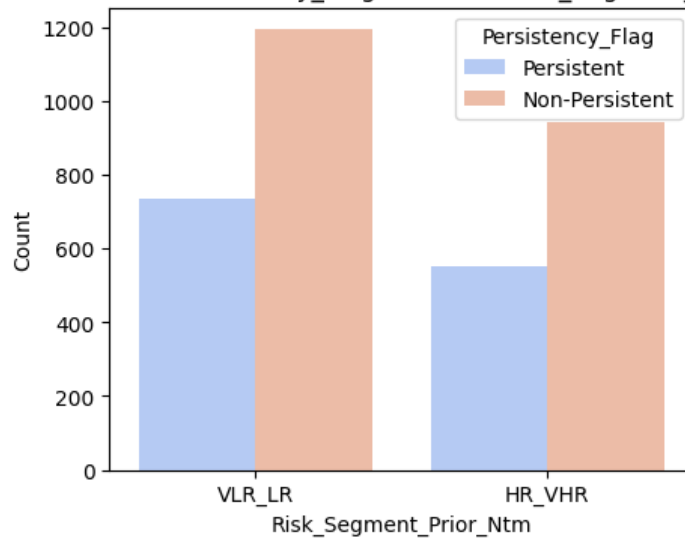
- The highest persistency and non-persistency were observed among general practitioners and rheumatologists.
- Non-specialists recorded more non-persistent patients overall.
- The OB/GYN/Others/PCP/Unknown group contributed the most prescriptions and corresponding results.

4.4 Risk Factors and Adherence Analysis

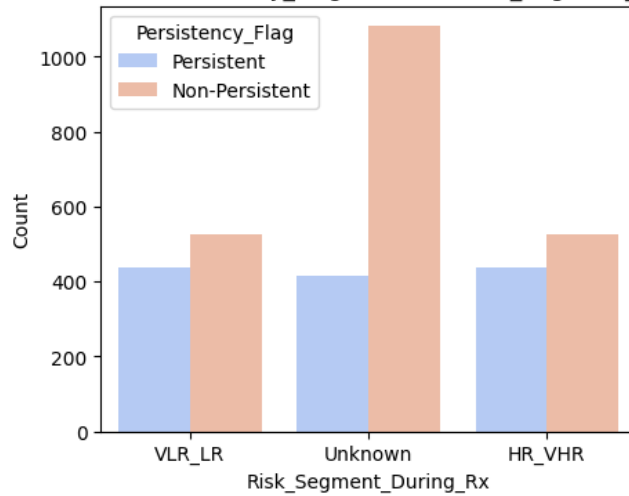
This section examined risk-related and adherence variables:

- Risk_Segment_Prior_Ntm and Risk_Segment_During_Rx

Distribution of Persistency_Flag based on Risk_Segment_Prior_Ntm

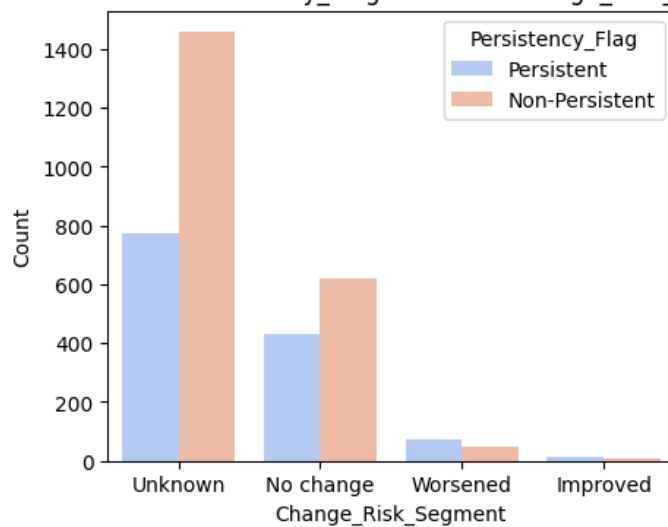


Distribution of Persistency_Flag based on Risk_Segment_During_Rx

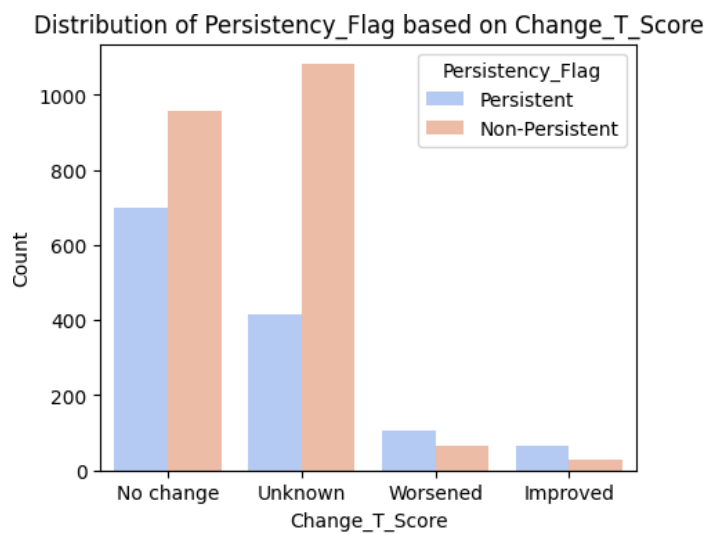


- Change_Risk_Segment

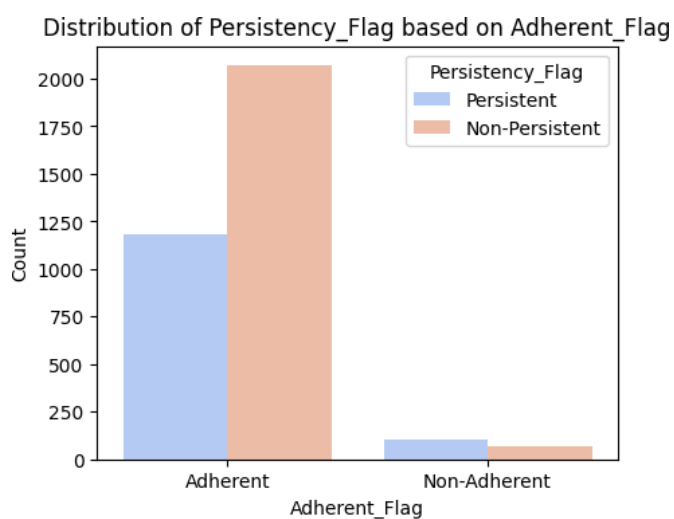
Distribution of Persistency_Flag based on Change_Risk_Segment



- Change_T_Score



- Adherent_Flag



Findings:

1. Non-persistence was most common in low-risk or unknown patient categories.
2. The majority of patients showed no significant change in their risk level or T-score during treatment.
3. Even among those marked as adherent, many did not remain persistent, suggesting external factors affect continuation.

5. Model Building and Evaluation

This section describes the Week 10 baseline model (Random Forest), which served as an initial benchmark before model comparison and final selection in Week 12.

After performing exploratory analysis, a predictive model was developed following these steps:

1. **Feature Engineering:** Refined and selected the most relevant variables.

- Feature Engineering using demographics

```
# Feature Engineering using demographics
data['IsFemale'] = data['Gender'].apply(lambda gender: 1 if gender == 'Female' else 0)
data['IsCaucasian'] = data['Race'].apply(lambda race: 1 if race == 'Caucasian' else 0)
data['IsNonHispanic'] = data['Ethnicity'].apply(lambda ethnicity: 1 if ethnicity == 'Not Hispanic' else 0)
data['IsAgeGroup1'] = data['Age_Bucket'].apply(lambda age_bucket: 1 if age_bucket == '>75' else 0)
data['IsAgeGroup2'] = data['Age_Bucket'].apply(lambda age_bucket: 1 if age_bucket == '65-75' else 0)
data['IsAgeGroup3'] = data['Age_Bucket'].apply(lambda age_bucket: 1 if age_bucket == '55-65' else 0)
```

- Feature Engineering using physician analysis

```
# Feature Engineering using physician analysis
data['IsGeneralPractitioner'] = data['Ntm_Specialty'].apply(lambda specialty: 1 if specialty == 'GENERAL PRACTITIONER' else 0)
data['IsNonSpecialist'] = data['Ntm_Specialist_Flag'].apply(lambda flag: 1 if flag == 'Others' else 0)
data['IsOBGYNorPCP'] = data['Ntm_Specialty_Bucket'].apply(lambda bucket: 1 if bucket == 'OB/GYN/Others/PCP/Unknown' else 0)
```

- Feature Engineering using risk factors and adherence analysis

```
# Feature Engineering using risk factors and adherence analysis
data['IsLowRiskPrior'] = data['Risk_Segment_Prior_Ntm'].apply(lambda risk: 1 if risk == 'VLR_LR' else 0)
data['IsLowRiskDuring'] = data['Risk_Segment_During_Rx'].apply(lambda risk: 1 if risk == 'Unknown' else 0)
data['IsChangeRiskUnknown'] = data['Change_Risk_Segment'].apply(lambda change: 1 if change == 'Unknown' else 0)
data['IsChangeTScoreUnknown'] = data['Change_T_Score'].apply(lambda change: 1 if change == 'Unknown' or 'No Change' else 0)
data['IsAdherent'] = data['Adherent_Flag'].apply(lambda flag: 1 if flag == 'Adherent' else 0)
```

2. **Encoding:** Converted all necessary categorical fields for model training.

- Encoding categorical columns

```
# Apply LabelEncoder to categorical columns
label_encoder = LabelEncoder()

data['Region'] = label_encoder.fit_transform(data['Region'])
data['Ntm_Specialty'] = label_encoder.fit_transform(data['Ntm_Specialty'])
data['Persistency_Flag'] = label_encoder.fit_transform(data['Persistency_Flag'])

features = ['IsFemale', 'IsCaucasian', 'IsNonHispanic', 'IsAgeGroup1', 'IsAgeGroup2',
            'IsAgeGroup3', 'IsGeneralPractitioner', 'IsNonSpecialist', 'IsOBGYNorPCP',
            'IsLowRiskPrior', 'IsLowRiskDuring', 'IsChangeRiskUnknown', 'IsChangeTScoreUnknown', 'IsAdherent']
```

3. **Data Splitting:** Divided the dataset into training and test subsets.

- Identifying X and Y for model building

```
X = data[features]
y = data['Persistency_Flag']
```

- Splitting the dataset into train and test

```
# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. **Model Creation:** Built and trained a classification model.

- Creating, training and testing the model

```
# Create and train the model (Random Forest Classifier)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
print("X_train", X_train.shape)
print("y_train", y_train.shape)
print("X_test", X_test.shape)
print("y_test", y_test.shape)
```

```
X_train (2739, 14)
y_train (2739,)
X_test (685, 14)
y_test (685,)
```

5. **Evaluation:** Analyzed performance using a confusion matrix and accuracy metrics.

Building the Model

```
# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.635036496350365
```

Evaluating the Model using the confusion matrix

```
from sklearn.metrics import confusion_matrix
# Evaluate the model
class_names = label_encoder.classes_

print(confusion_matrix(y_test, y_pred))
```

```
[[366  65]
 [183  71]]
```

Calculating and visualizing accuracy

```
from sklearn.metrics import accuracy_score, recall_score

# Accuracy Score
Accuracy = accuracy_score(y_test, y_pred)
print('Accuracy Score:', Accuracy)

# Precision Score
Precision = precision_score(y_test, y_pred)
print('Precision Score:', Precision)

# True positive Rate (TPR) or Sensitivity or Recall
TPR = recall_score(y_test, y_pred)
print('True positive Rate:', TPR)

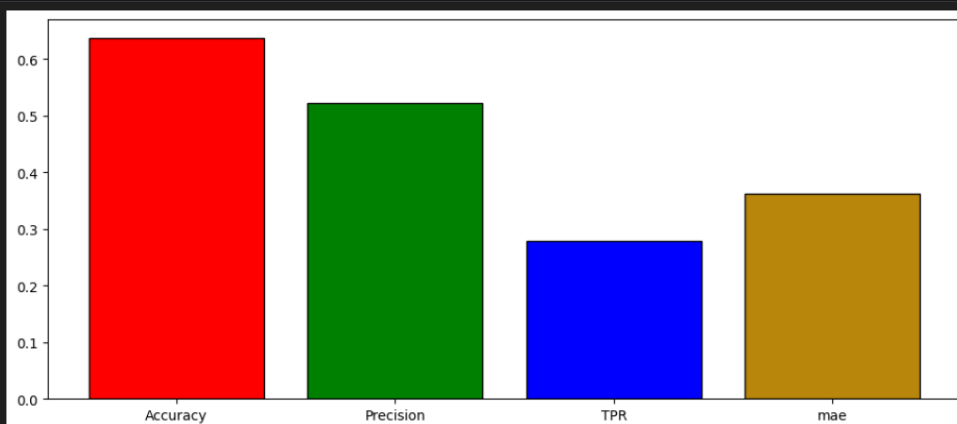
# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)
print("Mean Absolute Error:", mae)
```

```
Accuracy Score: 0.637956204379562
Precision Score: 0.5220588235294118
True positive Rate: 0.2795275590551181
Mean Absolute Error: 0.362043795620438
```

```
plt.figure(figsize = (12, 5))

result = [Accuracy, Precision, TPR, mae]
label = ["Accuracy", "Precision", "TPR", "mae"]
colors = ['red', 'green', 'blue', 'darkgoldenrod', 'orange']

plt.bar(label, result, color = colors, edgecolor='black')
plt.show()
```



6. **Accuracy Result:** The initial baseline model achieved 63.7% accuracy. This baseline was later compared against other model families during the Week 12 model selection process. This baseline served as the reference point during the expanded comparison conducted in Week 12.
7. **Pickle Storage:** Saved the trained model for reuse.

^ Saving the model using pickle

```
import pickle
from sklearn.ensemble import RandomForestClassifier

model_filename = 'persistency_model.pkl'
with open(model_filename, 'wb') as model_file:
    pickle.dump(model, model_file)

with open(model_filename, 'rb') as model_file:
    loaded_model = pickle.load(model_file)
```

8. **Testing:** Verified model predictions on unseen data.

Creating a list of features to test saved model

```
# Define the list of features
features = ['IsFemale', 'IsCaucasian', 'IsNonHispanic', 'IsAgeGroup1', 'IsAgeGroup2',
            'IsAgeGroup3', 'IsGeneralPractitioner', 'IsNonSpecialist', 'IsOBGYNorPCP',
            'IsLowRiskPrior', 'IsLowRiskDuring', 'IsChangeRiskUnknown', 'IsChangeTScoreUnknown', 'IsAdherent']

# Create an empty dictionary to store feature values
data_dict = {}

# Prompt the user to input feature values
print("Please provide the following feature values:")

for feature in features:
    if feature == 'IsFemale':
        value = int(input("If patient is female, enter 1. If male, enter 0: "))
    elif feature == 'IsCaucasian':
        value = int(input("If patient caucasian, enter 1. If not, enter 0: "))
    elif feature == 'IsNonHispanic':
        value = int(input("If patient is Hispanic, enter 1. If not, enter 0: "))
    elif feature == 'IsAgeGroup1':
        value = int(input("If patient age is >75, enter 1. If not, enter 0: "))
    elif feature == 'IsAgeGroup2':
        value = int(input("If patient age is between 65-75, enter 1. If not, enter 0: "))
    elif feature == 'IsAgeGroup3':
        value = int(input("If patient is age is between 55-65, enter 1. If not, enter 0: "))
    elif feature == 'IsGeneralPractitioner':
        value = int(input("If patient's speciality is general practitioner, enter 1. If not, enter 0: "))
    elif feature == 'IsNonSpecialist':
        value = int(input("If patient's physician is a Non specialist, enter 1. If not, enter 0: "))
    elif feature == 'IsOBGYNorPCP':
        value = int(input("If patient's provider is an Obstetrics or Gynecology, enter 1. If not, enter 0: "))
    elif feature == 'IsLowRiskPrior':
        value = int(input("If patient's Risk_Segment_Prior_Ntm is VLR_LR, enter 1. If not, enter 0: "))
    elif feature == 'IsLowRiskDuring':
        value = int(input("If patient's Risk_Segment_During_Rx is Unknown, enter 1. If not, enter 0: "))
    elif feature == 'IsChangeRiskUnknown':
        value = int(input("If patient's Change_Risk_Segment is Unknown, enter 1. If not, enter 0: "))
    elif feature == 'IsChangeTScoreUnknown':
        value = int(input("If patient's Change_T_Score is Unknown or No change, enter 1. If not, enter 0: "))
    elif feature == 'IsAdherent':
        value = int(input("If patient's Adherent_Flag is Adherent, enter 1. If not, enter 0: "))
    elif feature.startswith('Is'):
        # For other binary features, prompt for a binary value (0 or 1)
        value = int(input(f"If Patient is {feature} enter 1 else enter 0: "))
    else:
        # For other features, prompt for a value
        value = input(f"Enter value for {feature}: ")
    data_dict[feature] = value
```

Testing the model

```
# Load the trained model from file
with open(model_filename, 'wb') as model_file:
    pickle.dump(model, model_file)

# Make predictions using the loaded model
prediction = loaded_model.predict(user_input_data[features])

# Display the prediction
if prediction[0] == 1:
    print("Prediction: Persistent")
else:
    print("Prediction: Non-Persistent")
```

Prediction: Non-Persistent

6. Model Selection and Comparison

To extend the baseline model from previous weeks, three different model families were evaluated in order to identify the most appropriate choice for predicting patient drug persistency. The goal was not only to find the model with the highest numerical performance, but also to select a solution that fits the business and clinical requirements of the pharmaceutical context, where interpretability and trust are essential.

The models evaluated were:

- **Logistic Regression (Linear Model)** - a transparent, explainable model that provides clear insights into how each feature affects persistency likelihood.
- **Random Forest (Ensemble Model)** - a more complex, non-linear model capable of capturing interactions between features.
- **Gradient Boosting (Boosting Model)** - a sequential, highly flexible model that often excels in structured prediction tasks.

Each model was trained on the same processed dataset and evaluated using Accuracy, Precision, Recall, and F1-score to provide a balanced comparison.

Model Performance Results

Model	Accuracy	Precision	Recall	F1
Logistic Regression (linear)	0.6204	0.4901	0.5866	0.5341
Random Forest (ensemble)	0.6350	0.5138	0.2913	0.3718
Gradient Boosting (boosting)	0.6394	0.5309	0.2362	0.3270

Detailed Interpretation of Results

Although Random Forest and Gradient Boosting achieved slightly higher accuracy scores, their Recall values were extremely low. This means they correctly identified fewer patients who would actually remain persistent. In a healthcare setting, this matters:

- A model with low Recall incorrectly assumes patients *won't* be persistent even when they will be,
- leading to unnecessary intervention strategies, misallocation of resources, and reduced confidence in the predictive system.

Logistic Regression, on the other hand, achieved the strongest Recall and the best F1-score.

This indicates:

- It catches more “true persistent” patients,
- It offers a more stable balance between sensitivity and precision,
- It makes fewer harmful misclassifications in the direction that matters most for the business.

Business Interpretation & Strategic Fit

For a pharmaceutical company analyzing drug persistency, the model must support actionable decision-making. Clinical decision teams, physicians, and compliance specialists need to understand why a prediction was made.

This introduces critical constraints:

- High interpretability is required due to regulatory expectations and clinical responsibility.
- Explainability improves stakeholder trust and increases model adoption.
- Insights into feature importance guide patient engagement strategies, adherence interventions, and targeted communication.

While tree-based methods often perform well, they operate as black-box models, which is not ideal when predictions influence real-world health planning or patient outreach decisions.

Logistic Regression provides:

- Transparent coefficients
- Clear directionality (e.g., which factors increase or reduce persistency)
- Easy communication to non-technical stakeholders
- Simple auditability from a compliance and legal standpoint

From a business perspective, this makes it significantly more suitable, even if the raw accuracy is slightly lower.

Final Model Selection

Considering the performance metrics and the business constraints of the healthcare industry, Logistic Regression was selected as the final model. It offers:

- The best overall balance between Recall and F1
- A much lower risk of missing persistent patients
- Full interpretability for medical and business teams
- Alignment with industry expectations for transparent AI use

The selected model was saved as a .pkl file and is ready for use in further evaluation, deployment, or integration into future patient support dashboards.

Conclusion

This project showed that patient demographics, physician characteristics, and risk or adherence signals all influence whether someone stays on their medication. The Week 10 model gave us a solid baseline, but the deeper model comparison in Week 12 made it clear that accuracy alone isn't enough - especially in healthcare.

Although the ensemble and boosting models performed slightly better on paper, they struggled to identify persistent patients and were harder to explain. Logistic Regression offered the best balance: stronger recall, a more reliable F1-score, and clear interpretability. This makes it a better fit for real-world use, where medical teams need to trust and understand the predictions.

With more data and additional feature engineering, the model could be made even stronger. But as it stands, this work provides a practical starting point for improving patient engagement and supporting more targeted adherence strategies.

GitHub Repository Link

https://github.com/magdalena-ivanova-ds/Data-Glacier-Virtual-Internship/tree/main/week13_Final_Project